

Giới thiệu về Đệ quy (Recursion)

Đệ quy là kỹ thuật trong đó một hàm hoặc thủ tục gọi lại chính nó. Một thuật toán đệ quy thường có:

- **Trường hợp cơ sở (Base case):** Điều kiện dừng, không gọi đệ quy.
- **Bước đệ quy (Recursive step):** Chia bài toán thành bài toán nhỏ hơn cùng dạng và gọi lại chính nó.

Các sơ đồ thuật toán đệ quy phổ biến:

1. Đệ quy tuyến tính: Mỗi lần gọi chỉ gọi lại một lần (VD: giai thừa).
2. Đệ quy nhánh: Mỗi lần gọi có thể gọi lại nhiều lần (VD: Tháp Hà Nội, Fibonacci).
3. Đệ quy đuôi (tail recursion): Lời gọi đệ quy là thao tác cuối cùng trong hàm.

Phương pháp giải hệ thức truy hồi (Recurrence Relations)

Có ba phương pháp chính:

1. **Phương pháp thế (Substitution method):** Đoán nghiệm và chứng minh bằng quy nạp.
2. **Phương pháp cây đệ quy (Recursion tree method):** Biểu diễn lời gọi đệ quy dạng cây, tính tổng chi phí.
3. **Định lý Master (Master Theorem):** Áp dụng cho dạng $T(n) = aT(n/b) + n^d$.

Định lý Master (Master Theorem)

Cho n là lũy thừa của b , xét hệ thức truy hồi:

$$T(n) = \begin{cases} 1 & \text{nếu } n \leq 1 \\ aT\left(\frac{n}{b}\right) + n^d & \text{nếu } n > 1, a \geq 1, b > 1, d \geq 0 \end{cases}$$

Khi đó:

$$T(n) = \begin{cases} O(n^d) & \text{nếu } a < b^d \\ O(n^d \log n) & \text{nếu } a = b^d \\ O(n^{\log_b a}) & \text{nếu } a > b^d \end{cases}$$

Ví dụ 1

Xét hệ thức:

$$T(n) = \begin{cases} 1 & \text{nếu } n \leq 0 \\ 2T(n-1) - 1 & \text{nếu } n > 0 \end{cases}$$

Giải: Đây không phải dạng Master. Giải bằng phương pháp thế:

$$\begin{aligned} T(0) &= 1 \\ T(1) &= 2T(0) - 1 = 2(1) - 1 = 1 \\ T(2) &= 2T(1) - 1 = 2(1) - 1 = 1 \\ T(n) &= 1 \quad \forall n \end{aligned}$$

Vậy $T(n) = O(1)$.

Ví dụ 2

Xét hệ thức:

$$T(n) = \begin{cases} 1 & \text{nếu } n \leq 0 \\ 3T(n-1) & \text{nếu } n > 0 \end{cases}$$

Giải:

$$\begin{aligned} T(0) &= 1 \\ T(1) &= 3T(0) = 3 \\ T(2) &= 3T(1) = 3^2 \\ T(n) &= 3^n \end{aligned}$$

Vậy $T(n) = O(3^n)$.

Ví dụ 3

1. $T(1) = 1$, $T(n) = 2T(n/2) + 6n - 1$ với n là lũy thừa của 2

Áp dụng Master Theorem: $a = 2$, $b = 2$, $d = 1$ (vì $6n - 1 = O(n^1)$). Ta có $b^d = 2^1 = 2$, $a = 2$, suy ra $a = b^d$. Vậy $T(n) = O(n^d \log n) = O(n \log n)$.

2. $T(1) = 2$, $T(n) = 4T(n/3) + 3n - 5$ với n là lũy thừa của 3

$a = 4$, $b = 3$, $d = 1$. $b^d = 3$, $\log_b a = \log_3 4 \approx 1.2619$. Vì $a > b^d$ ($4 > 3$) $\Rightarrow T(n) = O(n^{\log_3 4})$.

3. $T(1) = 1$, $T(n) = 3T(n/2) + n^2 - n$ với n là lũy thừa của 2

$a = 3$, $b = 2$, $d = 2$. $b^d = 2^2 = 4$. $\log_b a = \log_2 3 \approx 1.585$. Vì $a < b^d$ ($3 < 4$) $\Rightarrow T(n) = O(n^d) = O(n^2)$.

Bài tập 1 (Giải chi tiết 4 hệ thức truy hồi)

Bài 1: $T(1) = 1$, $T(n) = 3T(n-1) + 2$

Giải bằng phương pháp thế:

$$\begin{aligned} T(1) &= 1 \\ T(2) &= 3T(1) + 2 = 3 + 2 = 5 \\ T(3) &= 3T(2) + 2 = 15 + 2 = 17 \\ T(4) &= 3T(3) + 2 = 51 + 2 = 53 \end{aligned}$$

Dạng tổng quát: $T(n) = 3^{n-1} + (3^{n-2} + \dots + 3^0) \cdot 2$

$$T(n) = 3^{n-1} + 2 \cdot \frac{3^{n-1} - 1}{2} = 2 \cdot 3^{n-1} - 1$$

Vậy $T(n) = O(3^n)$.

Bài 2: $T(1) = 3$, $T(n) = T(n-1) + 2n - 3$

$$\begin{aligned} T(n) &= T(1) + \sum_{k=2}^n (2k - 3) = 3 + 2 \sum_{k=2}^n k - 3 \sum_{k=2}^n 1 \\ &= 3 + 2 \left(\frac{n(n+1)}{2} - 1 \right) - 3(n-1) \\ &= 3 + n(n+1) - 2 - 3n + 3 = n^2 + n + 4 - 3n = n^2 - 2n + 4 \end{aligned}$$

Vậy $T(n) = O(n^2)$.

Bài 3: $T(1) = 1, T(n) = 2T(n-1) + n - 1$

$$\begin{aligned}T(1) &= 1 \\T(2) &= 2(1) + 1 = 3 \\T(3) &= 2(3) + 2 = 8 \\T(4) &= 2(8) + 3 = 19\end{aligned}$$

Dự đoán: $T(n) = 2^n - n - 1$. Thử $n = 1$: $2 - 1 - 1 = 0$ (sai). Thử lại: $T(n) = 2^{n+1} - n - 2$. Với $n = 1$: $4 - 1 - 2 = 1$ đúng. Chứng minh bằng quy nạp: Giả sử đúng với $n-1$: $T(n) = 2(2^n - (n-1) - 2) + n - 1 = 2^{n+1} - 2n + 2 + n - 1 = 2^{n+1} - n + 1$? Chưa khớp. Kiểm tra kỹ: $T(n) = 2^{n+1} - n - 2$: Với $n = 2$: $2^3 - 2 - 2 = 8 - 4 = 4$ (sai vì $T(2) = 3$). Vậy $T(n) = 2^{n+1} - n - 3$: $n = 2$: $8 - 2 - 3 = 3$ đúng. Vậy $T(n) = O(2^n)$.

Bài 4: $T(1) = 4, T(n) = 2T(n/2) + 3n + 2$ (n là lũy thừa của 2)

Áp dụng Master Theorem: $a = 2, b = 2, d = 1, b^d = 2, a = 2 \Rightarrow a = b^d$. Vậy $T(n) = O(n^d \log n) = O(n \log n)$.

Bài tập bổ sung (giải nhanh theo Master Theorem)

- $T(n) = 6T(n/6) + 2n + 3$: $a = 6, b = 6, d = 1 \Rightarrow a = b^d \Rightarrow O(n \log n)$.
- $T(n) = 6T(n/6) + 3n - 1$: tương tự $O(n \log n)$.
- $T(n) = 4T(n/3) + 2n - 1$: $a = 4, b = 3, d = 1, b^d = 3, a > 3 \Rightarrow O(n^{\log_3 4})$.
- $T(n) = 3T(n/2) + n^2 - 2n + 1$: $a = 3, b = 2, d = 2, b^d = 4, a < 4 \Rightarrow O(n^2)$.
- $T(n) = 3T(n/2) + n - 2$: $a = 3, b = 2, d = 1, b^d = 2, a > 2 \Rightarrow O(n^{\log_2 3})$.

Bài tập 2: Độ quy và Khử đệ quy

1. In biểu diễn nhị phân của một số nguyên

Giải thuật đệ quy

Algorithm 1 In nhị phân đệ quy

InNhiPhanRecn $n > 1$ InNhiPhanRecn/2 In $n \% 2$

Giải thuật khử đệ quy (dùng stack)

Algorithm 2 In nhị phân khử đệ quy

InNhiPhanKhun Khởi tạo stack rỗng $n > 0$ Push $n \% 2$ vào stack $n \leftarrow n/2$ stack không rỗng In pop()

Đánh giá và so sánh

- Cả hai đều có độ phức tạp $O(\log n)$.
- Đệ quy: ngắn gọn, dễ hiểu nhưng tốn bộ nhớ stack.
- Khử đệ quy: dài hơn nhưng kiểm soát bộ nhớ tốt hơn.

2. Phân tích số nguyên thành thừa số nguyên tố

Giải thuật đệ quy

Algorithm 3 Phân tích thừa số đệ quy

PhanTichRecn, $u \ n == 1 \ i \leftarrow u \ n \ n \% i == 0 \text{ In } i \text{ PhanTichRecn}/i, i$

Giải thuật khử đệ quy

Algorithm 4 Phân tích thừa số khử đệ quy

PhanTichKhun $i \leftarrow 2 \ i \cdot i \leq n \ n \% i == 0 \text{ In } i \ n \leftarrow n/i \ i \leftarrow i + 1 \ n > 1 \text{ In } n$

3. Tìm số Fibonacci thứ n

Giải thuật đệ quy

Algorithm 5 Fibonacci đệ quy

FiboRecn $n \leq 1 \ n \text{ FiboRecn} - 1 + \text{FiboRecn} - 2$

Độ phức tạp: $O(2^n)$.

Giải thuật khử đệ quy (quy hoạch động)

Algorithm 6 Fibonacci khử đệ quy

FiboKhun $n \leq 1 \ n \ a \leftarrow 0, b \leftarrow 1 \ i \leftarrow 2 \ n \ c \leftarrow a + b \ a \leftarrow b \ b \leftarrow c \ b$

Độ phức tạp: $O(n)$.

4. Bài toán Tháp Hà Nội

Giải thuật đệ quy

Algorithm 7 Tháp Hà Nội đệ quy

ThapHaNoiRecn, $A, B, C \ n == 1 \text{ In "Chuyển đĩa 1 từ } A \text{ sang } C"$ ThapHaNoiRecn - 1, $A, C, B \text{ In "Chuyển đĩa } n \text{ từ } A \text{ sang } C"$ ThapHaNoiRecn - 1, B, A, C

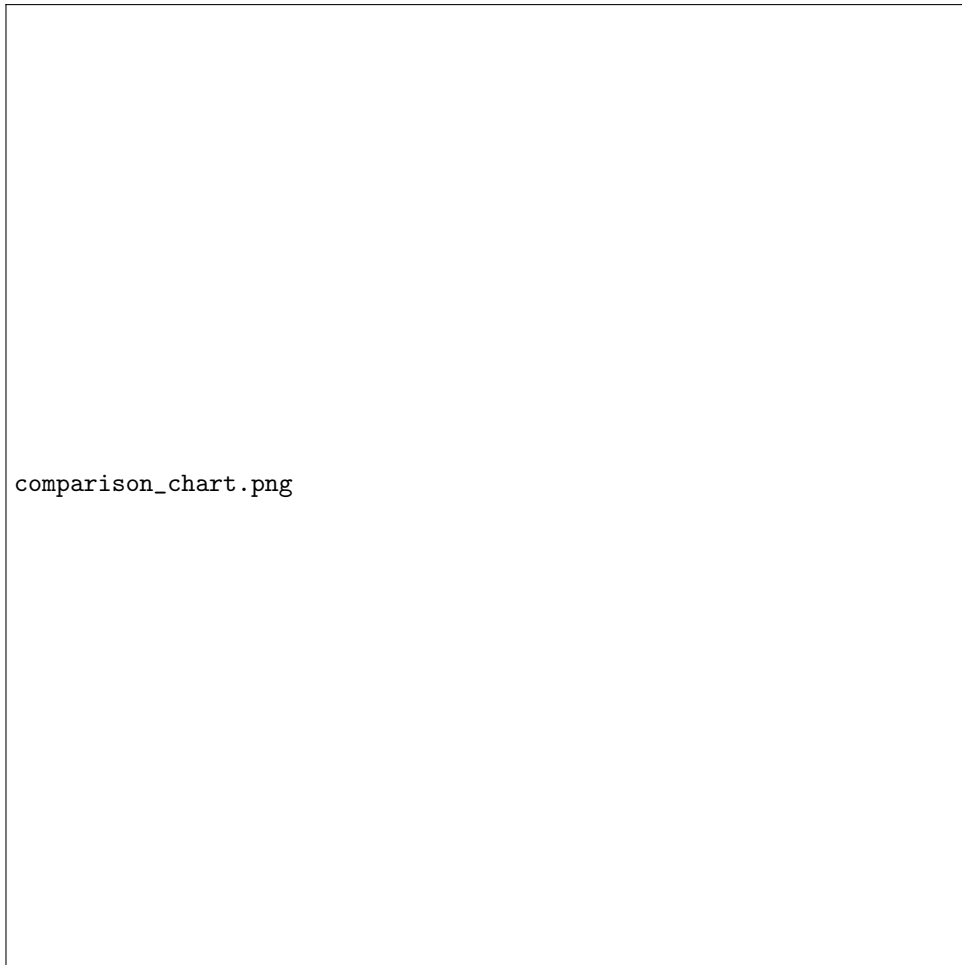
Độ phức tạp: $O(2^n)$.

Giải thuật khử đệ quy (dùng stack)

Dùng stack để mô phỏng lời gọi đệ quy. Độ phức tạp vẫn $O(2^n)$ nhưng tốn thêm bộ nhớ stack.

Bảng so sánh thời gian chạy

Bài toán	Đệ quy	Khử đệ quy	Ghi chú
In nhị phân	$O(\log n)$	$O(\log n)$	Khử đệ quy dùng stack
Phân tích thừa số	$O(\sqrt{n})$	$O(\sqrt{n})$	Khử đệ quy hiệu quả hơn
Fibonacci	$O(2^n)$	$O(n)$	Khử đệ quy tốt hơn nhiều
Tháp Hà Nội	$O(2^n)$	$O(2^n)$	Đệ quy tự nhiên hơn



Bài tập 3: Xây dựng bài toán, thiết kế thuật toán, phân tích và cài đặt

Bài toán 1: Bài toán cái túi (Knapsack Problem)

Phát biểu bài toán

Cho n đồ vật, mỗi đồ vật i có trọng lượng P_i và giá trị V_i . Một cái túi có thể chứa tối đa trọng lượng M . Hãy chọn các đồ vật để tổng giá trị là lớn nhất.

Thuật toán đệ quy (Quy hoạch động)

Gọi $F(i, w)$ là giá trị lớn nhất có thể đạt được khi xét từ đồ vật 1 đến i với trọng lượng tối đa w .

$$F(i, w) = \max(F(i-1, w), V_i + F(i-1, w - P_i)) \quad \text{nếu } w \geq P_i$$

Base case: $F(0, w) = 0$.

Algorithm 8 Knapsack đệ quy

$\text{Knapsack}(i, w) ::= 0$ OR $w == 0$ 0 $P[i] > w$ $\text{Knapsack}(i-1, w)$ $a \leftarrow \text{Knapsack}(i-1, w)$ $b \leftarrow V[i] + \text{Knapsack}(i-1, w - P[i])$ $\max(a, b)$

Chứng minh tính đúng đắn

Bằng quy nạp trên i : Giả sử $F(i-1, w)$ đúng với mọi w . Với đồ vật i , hoặc không chọn (giá trị $F(i-1, w)$), hoặc chọn (giá trị $V_i + F(i-1, w - P_i)$). Lấy max của hai trường hợp.

Độ phức tạp

Đệ quy: $O(2^n)$. Dùng quy hoạch động: $O(n \cdot M)$.

Bài toán 2: Chia thưởng (Reward Distribution Problem)

Phát biểu bài toán

Có m phần thưởng giống nhau, chia cho n học sinh xếp hạng $1 \dots n$ sao cho học sinh hạng cao nhận không ít hơn học sinh hạng thấp. Tìm số cách chia.

Thuật toán đệ quy

Gọi $f(m, n, k)$ là số cách chia m phần thưởng cho n học sinh, mỗi học sinh nhận ít nhất k phần thưởng.

$$f(m, n, k) = \begin{cases} 1 & \text{nếu } m = nk \\ 0 & \text{nếu } m < nk \\ \sum_{i=k}^m f(m-i, n-1, i) & \text{nếu } n > 1 \end{cases}$$

Algorithm 9 Chia thưởng đệ quy

```
ChiaThuong(m, n, k) m == n * k 1 m < n * k 0 n == 1 1 count ← 0 i ← k m count ←  
count + ChiaThuong(m - i, n - 1, i) count
```

Chứng minh tính đúng đắn

Base case: Nếu $n = 1$, chỉ có 1 cách (học sinh duy nhất nhận hết). Nếu $m = nk$, chỉ có 1 cách (mỗi học sinh nhận đúng k). Bước đệ quy: học sinh đầu nhận i ($i \geq k$), số còn lại chia cho $n - 1$ học sinh với mỗi người nhận $\geq i$.

Độ phức tạp

$O(m^n)$ trong trường hợp xấu nhất.

Cài đặt minh họa bằng Python

```
# Knapsack - Quy hoạch động khử đệ quy
def knapsack_dp(P, V, M):
    n = len(P)
    dp = [[0]*(M+1) for _ in range(n+1)]
    for i in range(1, n+1):
        for w in range(1, M+1):
            if P[i-1] <= w:
                dp[i][w] = max(dp[i-1][w], V[i-1] + dp[i-1][w-P[i-1]])
            else:
                dp[i][w] = dp[i-1][w]
    return dp[n][M]

# Chia thưởng - đệ quy có nhớ
def chia_thuong(m, n, k, memo={}):
    if (m, n, k) in memo: return memo[(m, n, k)]
    if m == n * k: return 1
    if m < n * k: return 0
    if n == 1: return 1
    total = 0
    for i in range(k, m+1):
        total += chia_thuong(m-i, n-1, i, memo)
    memo[(m, n, k)] = total
    return total
```